Scripting Languages - Lecture 7: Text processing (4 Nov)

Lecture overview

Lecture 7 introduces advanced text processing techniques that move beyond basic Python string methods like `split`, which fail to handle punctuation, abbreviations, and complex token boundaries effectively. The session emphasizes preprocessing as a critical step involving tokenization, sentence detection, and normalization (lemmatization) to convert raw text into structured data suitable for analysis. While manual implementation is possible, the lecture advocates using established libraries such as NLTK and spaCy, which employ evolved algorithms or machine learning models to accurately resolve ambiguous cases like "Mr." or contractions. These tools enable robust exploration of textual data through concordances, similarity analysis, common contexts, and collocations, revealing statistically significant word associations and usage patterns across different corpora like Moby Dick and Sense and Sensibility.

The second half of the lecture addresses program reliability by covering errors, exceptions, and debugging strategies. It distinguishes between various Python exceptions-such as `SyntaxError`, `IndexError`, and `FileNotFoundError`-and explains how to interpret tracebacks to locate the root cause of failures. The session details a general debugging strategy involving logical analysis and intermediate print statements, followed by the implementation of `try-except-else` blocks to handle unforeseen runtime errors gracefully. By targeting specific exceptions rather than catching all errors broadly, developers can prevent program termination due to issues like encoding problems or invalid data types while ensuring that genuine coding faults remain visible for correction.

Slide 1: faculty of arts

This lecture introduces the seventh session on scripting languages within the Faculty of Arts, focusing specifically on text processing, errors, and exceptions. Delivered by Tim Van de Cruys on Wednesday, November 4, 2026, the session establishes the foundational context for exploring how scripts handle textual data and manage runtime issues.

Slide 2: Text processing

Text processing has so far relied on basic Python machinery, specifically the `str` object and its methods like `split`, `join`, `lower`, `strip`, `isalpha`, and `isnumeric`. For instance, `doc.split()` interprets a string as a list of words. However, these fundamental tools are limited and only function effectively up to a certain extent, indicating the need for more advanced approaches beyond simple string manipulation.

Slide 3: Splitting on whitespace

Splitting a string on whitespace using `s.split()` separates text only at spaces, leaving punctuation attached to adjacent words and keeping quoted phrases intact as single tokens. For example, splitting "Mr. J. Simpson said: 'Astounding!'" results in the list ["Mr.", "J.", "Simpson", "said:", "'Astounding!'"], demonstrating that colons remain with "said" and quotes are not removed. This method does not detect sentence boundaries or isolate punctuation marks from words.

Slide 4: Preprocessing

Preprocessing commonly organizes raw text through tokenization, sentence detection, and sometimes normalization such as lemmatization. The right steps depend on the analysis; they are useful tools rather than a strict requirement before every kind of text work. Language-aware modules handle many cases that a short hand-written split rule would miss.

Slide 5: Tokenization

Tokenization decides where to mark units in a character string. Spaces and punctuation are characters too, so a program needs a convention for whether punctuation, contractions, names, and other forms count as one token or several. This slide sets up that boundary problem; the Moby Dick token count appears later with the NLTK Text object.

Slide 6: Tokenization

Tokenization is the fundamental, automated task of splitting a text string into individual units called tokens, which include words, punctuation marks, and other elements. While various algorithms exist to perform this operation using cues like whitespace and punctuation, no single method is perfect. For instance, applying tokenization to the sentence '"Hark! A vagrant," he said.' correctly separates it into distinct components such as quotation marks, capitalized words, and individual punctuation symbols.

Slide 7: Tricky tokenization cases

The examples ask whether expressions such as “prime minister,” “New York,” and “I’m” should be treated as one token or more. There is no universal answer independent of purpose: tokenization conventions vary by language, tool, and downstream task.

Slide 8: Tricky tokenization cases

The slide adds possessives, hyphenated forms, abbreviations, emoticons, hashtags, URLs, and chemical names to the edge cases. Punctuation alone cannot settle all boundaries; a tokenizer uses a documented set of rules and may still make task-specific choices.

Slide 9: Sentence detection

Sentence detection distinguishes between unambiguous terminators like question and exclamation marks and the highly ambiguous period. While a final dot often ends a sentence, periods within abbreviations such as "U.N." or "etc." must not trigger segmentation. Because simple algorithms fail to differentiate these cases without extensive exception handling, context is essential for accurate detection, necessitating the use of evolved existing modules rather than custom rule-based approaches.

Slide 10: Normalization

Lemmatization maps inflected forms such as walk, walks, walked, and walking to a shared lemma, walk. It requires language-specific knowledge, especially for irregular forms. The slide notes that many analyses skip normalization for simplicity, and because the original forms can carry useful information; the NLTK example comes later.

Slide 11: Spacy & NLTK

NLTK and spaCy are separate Python NLP libraries. Both provide tools for preprocessing, and can support linguistic annotation and word-based analysis. Their components and methods differ, so choose and configure a pipeline for the language and task rather than assuming one library is always more accurate or faster.

Slide 12: NLTK

In this example, NLTK’s word_tokenize separates punctuation such as the colon and exclamation mark while keeping “Mr.” and “J.” together with their abbreviation periods. It preserves capitalization, and the opening quote remains attached to “Astounding” in this output; tokenizers are useful but not perfect.

Slide 13: NLTK

NLTK’s word_tokenize produces tokens, while sent_tokenize identifies sentence spans. The example preserves the abbreviations “Mr.” and “J.” within the first sentence instead of treating each period as a sentence boundary. Compared with split(), it handles punctuation and sentence structure more carefully, though the quote example still shows a limitation.

Slide 14: NLTK

NLTK's `WordNetLemmatizer` reduces words to their base lemma form, such as converting "rocks" to "rock" and "corpora" to "corpus." While the default behavior interprets input as a noun, specifying the part-of-speech argument (e.g., `pos="a"` for adjectives) is essential for accurate reduction of words like "better" to "good," preventing incorrect defaults.

Slide 15: NLTK: explore keywords in context

The NLTK Text object wraps a token list and adds exploratory methods: concordance, similar words, common contexts, and collocations. The code reads and tokenizes Moby Dick and Sense and Sensibility, then creates one Text object for each corpus. The Moby Dick file yields about 260,000 tokens; wrapping the token list as an NLTK Text object makes the exploration methods available.

Slide 16: Concordance

A concordance displays occurrences of a chosen word with nearby text on each side (keyword in context). It lets a reader inspect usage across a corpus, as with “monstrous” in Moby Dick; it does not automatically identify a word’s meaning or grammatical role.

Slide 17: Similar words

NLTK’s similar() method finds words that occur in similar contexts in a corpus. For “monstrous,” Moby Dick returns words such as “mystifying,” while Sense and Sensibility returns degree words such as “very” and “exceedingly.” These are contextual usage patterns, not synonyms, and the comparison motivates the different use Austen makes of the word.

Slide 18: Common contexts

common_contexts() compares the surrounding patterns for words in one corpus. The displayed patterns, such as a _ pretty and am _ glad, show places where “monstrous” and “very” can occur in similar grammatical frames. The method reports shared contexts; it does not claim the words have identical meanings.

Slide 19: Common contexts

For Sense and Sensibility, common_contexts() shows shared patterns such as a _ pretty, am _ glad, and a _ lucky. The concordance then lists the novel’s 11 uses of “monstrous,” including “a monstrous pretty girl.” Together the outputs let the reader inspect Austen’s use of “monstrous” as an intensifier.

Slide 20: Collocations

A collocation is a word sequence that occurs together more often than expected given the words’ individual frequencies. “Red wine” is a collocation in the slide’s example; “the wine” is not, because “the” appears with many words, so its co-occurrence with wine is less informative. The general idea is to compare observed co-occurrence with an expectation based on frequency.

Slide 21: Collocations: examples

The example collocations are corpus-specific: names and titles dominate both novels' outputs, such as "Captain Ahab" in Moby Dick and "Mrs. Jennings" in Sense and Sensibility. This shows what pairs are unusually frequent within these texts, not necessarily general-language collocations. The slide leaves the statistical measures for the practical session.

Slide 22: Spacy

spaCy loads a configured English pipeline with spacy.load(), then nlp(text) processes the string and returns a Doc object. The configured components can add token boundaries, sentence spans, lemmas, part-of-speech tags, and dependency relations. The exact components and model data depend on the installed pipeline; the slide does not identify its training corpus.

Slide 23: Spacy: tokenization and sentences

This slide demonstrates Spacy's tokenization and sentence processing by iterating through a document to print individual tokens. Using the example "Mr. J. Simpson said : ' Astounding ! '", it illustrates how the library breaks text into discrete units, separating punctuation marks like colons, quotes, and exclamation points from words and abbreviations.

Slide 24: Spacy: tokenization and sentences

spaCy stores sentence boundaries on the Doc and exposes them through doc.sents. In the example it keeps "Mr." and "J." inside the first sentence. spaCy's tokenizer uses language-specific rules and exceptions; other pipeline components may use statistical models, and the output can still have edge cases such as quote attachment.

Slide 25: Spacy: linguistic annotations

spaCy automatically annotates tokens with linguistic features such as `token.lemma_` for the base form (e.g., "said" becomes "say"), `token.pos_` for part-of-speech tags like PROPN or VERB, and `token.is_stop` to identify frequent words typically excluded from analysis. The library also provides access to syntactic dependencies (`token.dep_`) and character patterns (`token.shape_`). In the provided example, tokens like "Mr." and "Simpson" are correctly identified as proper nouns, while none are flagged as stop words.

Slide 26: A class for a text

This custom `Text` class extends previous versions by replacing simple string splitting with `nltk.word_tokenize` for accurate word lists and adding `nltk.sent_tokenize` to store sentences. It includes a `word_count` method returning the token list length and a `most_common` method utilizing Python's `Counter`-a specialized dictionary that automatically tallies item frequencies-to identify the top n words. This implementation ensures proper tokenization is used for analysis rather than basic testing approaches.

Slide 27: Using the class

The class now counts punctuation as tokens: the example's most common items include a comma and a quote. If an analysis should exclude punctuation, add an explicit filtering step in the constructor. A spaCy token's is_punct attribute can help when using spaCy tokens; NLTK string tokens need a suitable string-based filter.

Slide 28: A corpus of texts

A corpus of texts demonstrates composition by containing a list of `Text` objects. The `Corpus` class initializes by using `glob` to find files matching a pattern, sorting them, and instantiating a `Text` object for each filename. Its `word_count` method aggregates the total word count by iterating through all contained texts and summing their individual counts, as shown when creating a corpus from 'books/*.txt'.

Slide 29: Errors

Programs frequently fail due to unexpected issues like wrong input or missing files, which Python identifies as exceptions such as `FileNotFoundError`, `UnicodeDecodeError`, and `KeyError`. By default, these exceptions halt program execution, though developers often need to handle them to allow the code to continue. When errors arise from incorrect arguments or notation, it is safest to verify solutions online rather than relying solely on AI, marking the beginning of the debugging process.

Slide 30: Types of exceptions

The list gives common Python exceptions: SyntaxError for invalid syntax; NameError for an undefined name; IndexError for an out-of-range sequence index; KeyError for a missing mapping key; AttributeError for an absent object attribute; TypeError for an inappropriate operation or type; ValueError when the type is acceptable but the value is not; FileNotFoundError for a missing path; and UnicodeDecodeError for text that cannot be decoded with the chosen encoding. It is not exhaustive.

Slide 31: Tracebacks

A traceback is the chain of function calls leading to an error, designed to pinpoint the specific cause and location of a failure. In the provided example, calling `.lower()` on a list object (created by splitting a string) triggers an `AttributeError` because lists lack this method, whereas strings do. While simple tracebacks identify immediate issues like incorrect types, complex ones can reveal errors originating deep within nested modules or files, allowing developers to trace the exact function responsible for the crash.

Slide 32: Tracebacks

The bottom line of a traceback gives the exception type and message; the preceding frames show the call path. In this example, splitext fails inside os.path because it received a list rather than a path-like value. Read from the bottom to understand the error, then inspect the calls in your own code that supplied the invalid argument.

Slide 33: Debugging: general strategy

The general debugging strategy involves thinking carefully about the code's logic before inserting print statements. These statements are used to inspect intermediate steps and determine exactly what is happening during execution, allowing developers to trace the program flow and identify where errors occur.

Slide 34: Debugging: general strategy

Debugging requires thinking like a computer by analyzing each line of code to verify if the actual execution matches the intended logic. Use intermediate print statements to confirm inputs, outputs, and specific values at critical points, particularly checking if errors arise from special input cases. Since difficult problems often involve complex interactions, expect an iterative process of modifying code, re-executing, and chasing new errors until the issue is resolved.

Slide 35: Exception handling

While `if`-statements can prevent some exceptions, Python's `try-except-else` mechanism provides backup functionality to catch unforeseen errors and prevent program termination. The `try` block executes risky code, such as division, while the `except` block handles specific errors like `ZeroDivisionError`, and the `else` block runs only if no exception occurs. Crucially, developers should target unique exceptions rather than catching all general errors, ensuring that significant coding faults remain visible for proper debugging.

Slide 36: Exception handling

While `if`-statements can prevent some errors, Python's `try-except` mechanism catches unforeseen exceptions like `ZeroDivisionError` to prevent program termination. In the provided example where `i=10` and `j=0`, the `except` block executes backup code printing "cannot divide by zero" instead of crashing. Conversely, if the division succeeds without error, the `else` block runs to print the calculated result, ensuring the program handles both failure and success scenarios gracefully.

Slide 37: Exception handling

While appropriate if-statements can prevent some exceptions, programmers cannot always foresee every potential error, necessitating Python's built-in mechanism to catch them. This is demonstrated using a `try` block to attempt division, an `except ZeroDivisionError` clause to handle the specific case of dividing by zero, and an `else` block that executes only if no exception occurs, printing the successful result.

Slide 38: Try / except: a practical example

This practical example demonstrates using `try/except` to process multiple files where some may have encoding issues, such as non-Unicode text. By catching the specific `UnicodeDecodeError`, the script skips problematic files and continues processing the rest, preventing a single bad file from stopping the entire batch. However, broadly catching all exceptions is dangerous because it can hide real bugs; therefore, always target specific errors to ensure graceful failure without masking underlying code issues.

Slide 39: Try it: catching an error

The exercise tries to convert each string to an integer and catches ValueError for entries that cannot be converted, such as “twelve” and “3.5.” Successful conversions such as “12,” “7,” and “40” are appended; invalid entries are skipped. The example handles these particular positive values, rather than defining every string accepted by Python’s int().

Slide 40: Practical session / reading

Students are directed to complete Practical Session 6 by studying Chapter 1 of the NLTK book and accessing the lecture's code-along notebook on Toledo. The primary objective is to re-run the provided examples at home to reinforce understanding of scripting languages through hands-on practice.

Key points to remember

Reliable text analysis begins with an explicit tokenization and sentence-boundary convention. NLTK and spaCy provide language-aware tools, while their outputs still need to be checked for the task. NLTK’s concordances, similar-context results, common contexts, and collocations support corpus exploration; these are evidence about usage in the selected texts, not automatic definitions or synonyms. For program reliability, read tracebacks from the exception message through the call path, debug by inspecting inputs and outputs, and catch only the specific exceptions the program can recover from.